

# Distributed entertainment environment

S J Powers, M R Hinds and J Morphet

---

*Networked computer games have come of age in recent years, due to a combination of increased graphical sophistication and simplified networking in consumer level computer equipment.*

*This paper presents an overview of a new network game genre, the virtual environment based multi-user dungeon. It reviews the technical requirements necessary to deliver successful games of this type, and outlines a suggested architecture. Issues addressed in the architecture include scalability, persistence and consistency.*

---

## 1. Overview

The development of interactive computer games can be traced back to about 1960, with the development of Spacewar at MIT. However, it was not until 1972 that computer gaming penetrated the mass market, in the shape of Pong. Developed by Nolan Bushnell, founder of the first videogame company (Atari, 1968), it was an overnight sensation. Throughout the 1970s Atari continued to develop and release arcade-based gaming systems, and in 1976 it took videogames into consumers' homes, with the VCS 2600. Since that time, although the technology has drastically improved with the standard PC now a major player in the games market, the fundamental style of gaming has not changed.

While the latest Nintendo 64 console may boast the sort of graphics power previously only available on £50 000+ graphical workstations, the games themselves follow a very familiar pattern. The game designer sets certain goals (highscore, reaching a certain level, etc) together with certain tasks (avoid the monsters, collect the treasure, overtake the other cars, etc). Players follow an essentially directed path through the tasks, progressing from a starting point to a termination point, specified by the game designer. For each new game the player restores the game state to its initial conditions, and restarts along the same path.

This traditional style of gaming is now under threat, with the convergence of several games-related technologies, and a game genre known as MUDs (multi-user dungeons or multi-user dimensions).

MUDs offer the exception to the single player game described above [1]. Firstly, they are inherently multi-player, running over networks such as the Internet and on

bulletin boards (BBs). Players can join or leave the game at any time, and usually are able to play the same on-line character each time they play. Secondly, they are far less goal directed, offering a more organic, persistent playing environment, that can grow or shrink over time as it is modified by the users. Finally, while also offering tasks for players to complete, a large part of the game is made up of the interaction between players. This may manifest itself in either competition or co-operation, dependent on the game goals and the inclination of the players.

While these features of MUDs make them a highly compelling and interesting gaming environment, their userbase has thus far been small. One of the key reasons for this is their outdated, crude and simplistic client interface. Players interact through a text channel, receiving textual descriptions of their current location, and typing simple commands (e.g. 'take gold', 'go north', etc) to manipulate the environment.

With almost no professionally authored content, and a cliquey userbase, the MUD concept, as it currently stands, is doomed to a niche existence. However, this may be about to change, due to development in two key technical areas.

Firstly, increasing compute power has allowed complex 3-D virtual environments to be modelled and displayed. These environments allow players to both move through a virtual world and to interact with the entities found within it. Games such as Doom and Quake, utilising this style, have proved astonishingly popular, selling millions of copies and penetrating the cultural mainstream.

Secondly, networking facilities on both PCs and game consoles have become commonplace. With the Internet,

corporate LANs and dedicated gaming networks widely available, the network game has come of age. Most newly released games offer some form of network support, and some gaming companies are even making networking the primary form of play (e.g. LucasArts with X Wing versus Tie Fighter).

Combining these technologies with some of the ideas behind MUDs offers the potential for an exciting new game genre, the shared virtual environment MUD (VEMUD). Like a MUD, this is based around the idea of a persistent world, which exists and operates irrespective of the presence of players. Also like a MUD it utilises the idea of open-ended gameplay, with competition and co-operation between players to complete game goals. However, unlike a MUD, it draws on the most popular features of modern games; complex 3-D graphics and sophisticated sound effects. This allows for very intuitive interaction between player and environment, making the gaming experience both involving and of widespread appeal.

## 2. Virtual gaming environment requirements

In delivering any shared VE, be it a game or business tool, key technical issues are [2]:

- consistency,
- scalability,
- persistence.

A game-oriented VE also requires that the following three areas are addressed [3]:

- integrity,
- balance,
- extensibility.

### 2.1 Consistency

In a single player game, modelling a virtual environment is comparatively simple. The game maintains an internal model of the world which is repeatedly updated,

based on the internal game logic (triggering puzzles/traps, moving autonomous creatures, etc) and the input of the player (moving the viewpoint, selecting objects with which to interact, etc). After each update of the world a 2-D image of this 3-D model is rendered and presented to the player. This loop is repeated until the player wins, loses or gets bored (see Fig 1).

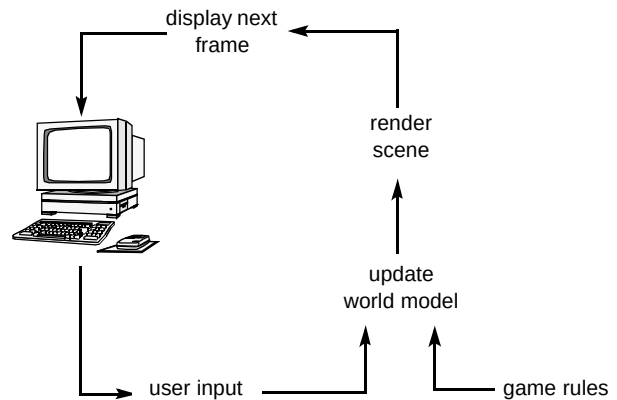


Fig 1 Single player game loop.

In a multiplayer version, the situation is greatly complicated by the need to ensure that each player appears to be interacting with the same environment (i.e. the same internal model). While each player will have a unique viewpoint on the environment, they must all experience the same changes to the environment in the same order.

The simplest means to achieve this is to centralise the model of the world and sequence each player's input into this model, while distributing the relevant changes to all players. This is the approach taken by MUDs, where a single central process sequences text input from each player in turn, calculates the changes to the world, and transmits a textual description of the new world state to the relevant players (see Fig 2).

While this ensures a consistent view of the environment for each player (by maintaining only a single model of the world), it is inappropriate for more complex graphical environments, as the link between the world model and the visual output consumes far more bandwidth than that

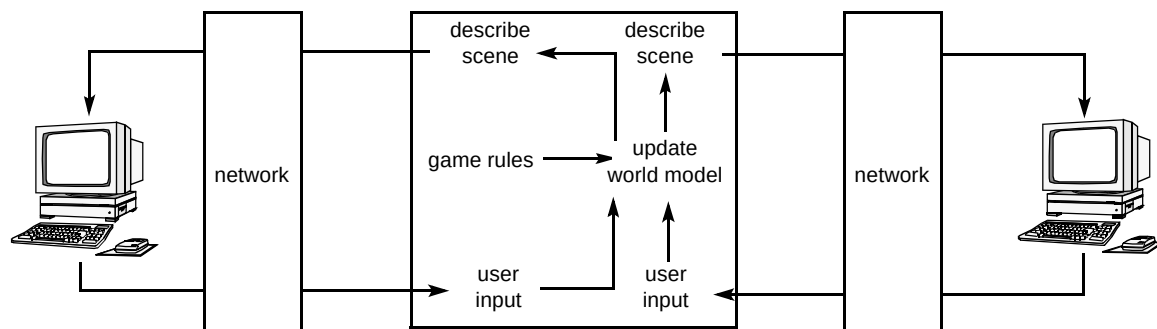


Fig 2 MUD game loop.

required by a text MUD. For a complex interactive 3-D environment it is necessary to present (as an approximate minimum) an image of  $320 \times 200$  pixels, 15 times a second. With 8 bits of colour depth per pixel this translates to approximately 1 Mbit/s of data. As a result, the approach taken by most games is both to allow each player to maintain a local model of the environment, with which he/she interacts, and to ensure that these local copies of the environment remain consistent with each other. This is shown in Fig 3.

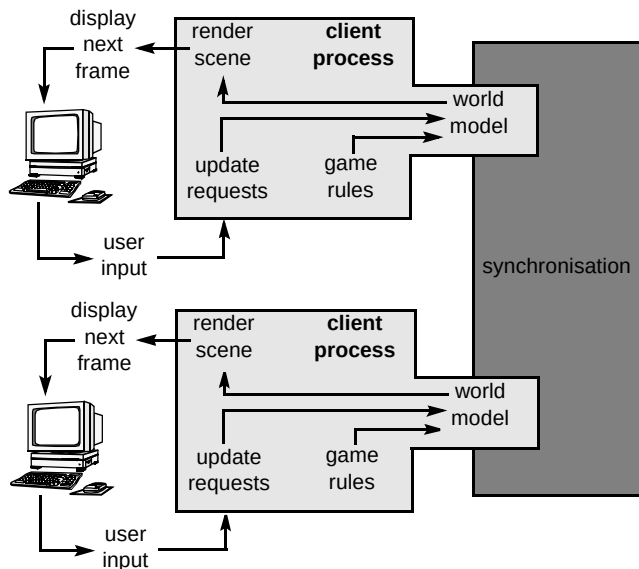


Fig 3 Synchronising shared environments.

## 2.2 Scalability

One possible approach to implementing the architecture shown in Fig 3, is to form peer-to-peer network connections for all clients, and alternate between updating the world and performing local rendering. This gives a simple system which effectively synchronises the clients around a repeating data exchange (see Fig 4).

This is the approach used by the popular game Doom, but it has two main drawbacks. Firstly, it links the state synchronisation to the rendering loop. This means that the speed of the screen update (a critical factor in producing an interactive environment) is linked to the speed of the slowest client network connection. This problem can be partially resolved by allowing clients to make local predictions about future world states, and then implementing roll-back to a previously agreed state if the predictions prove invalid. Duke Nukem, a later variation on the Doom genre, implements this approach.

The second and more serious drawback to this approach is that it lacks scalability. As the number of clients grows the amount of traffic each must send and receive grows,

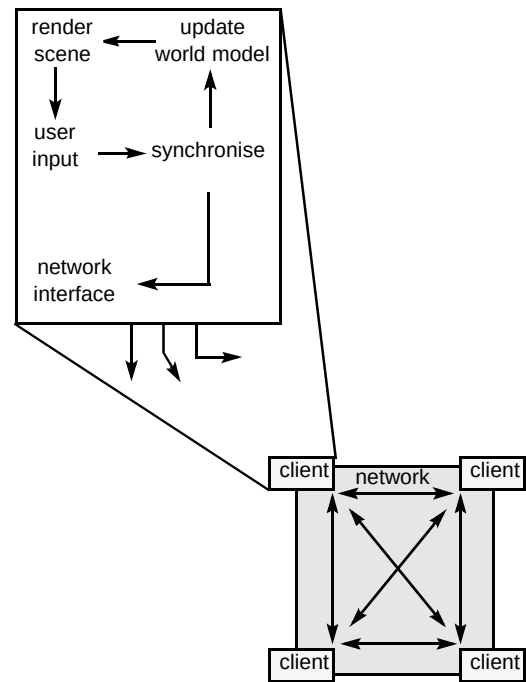


Fig 4 Client-to-client network architecture.

quickly overwhelming the available bandwidth to each client.

A possible solution to this problem is the provision of a central server, as shown in Fig 5.

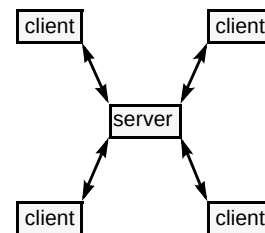


Fig 5 Server-oriented architecture.

In its simplest form the server acts as a broadcaster, sending the updates from each client to all other clients. This reduces the upstream traffic requirement from each client. A more sophisticated solution places a model of the world at the server, which both acts as the reference description of state and validates changes to this state, e.g. ensuring that when two players want to take an object only one of them succeeds.

It should be noted that this centralised world model does not replace the client-side models, but instead supplements them with a definitive state description.

Quake, the follow-up to Doom, uses this approach, allowing it to support up to 16 players, rather than the 4 of Doom.

The server can also reduce the downstream traffic to each client by only transmitting the world state changes that are currently relevant to each client's player. For example, a player will have no awareness of objects outside their current visual range. It is therefore unnecessary to keep the client informed of state changes to objects in this category.

However, while the central server approach offers improved scalability over the peer-to-peer client arrangement shown in Fig 4, it still has a potential bottleneck in the form of the server process. While network traffic to/from each client is reduced, the server process must cope with:

- all environment modification requests from each client,
- validating each request and calculating the new environment state,
- calculating which clients each environment modification affects,
- generating and sending the environment updates to all relevant clients.

With games involving hundreds of clients, moving objects and autonomous game creatures, a single central server will quickly become overwhelmed by both the network traffic and the processing required.

One approach to distributing this network traffic load is to divide the spatial environment into a number of pre-defined areas, and associate a multi-cast network address with each area. State changes to the environment are broadcast on the multi-cast address associated with the area in which the change occurred. Clients subscribe to the multi-cast address of the areas in which they are interested (i.e. the areas in visual range on the player's viewpoint), and hence receive all the state changes they require to maintain a valid model of the local environment. To move the player's avatar the client transmits the new position to the relevant multi-cast address, which in turn informs any other clients visualising the relevant area.

This approach, shown in Fig 6, is used in the NPSNET architecture, designed for military simulations [4].

### 2.3 Persistence

The NPSNET approach both helps to reduce the amount of information transmitted to each client and aids scalability by having no single central server. However, in the form shown in Fig 6, it lacks support for persistence.

Persistence is the ability of a virtual world to retain state over an extended period, so changes made by players persist between their gaming periods. Provision of persistence is a

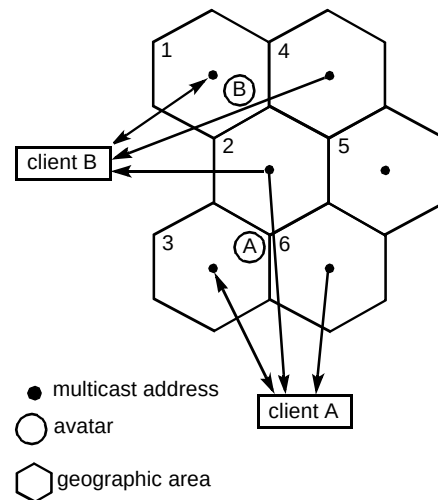


Fig 6 NPSNET approach to distributing network traffic.

critical difference between simple multi-player games and VE games. It is also a major factor in building an on-line community around a particular game, as it allows individuals to feel part of an evolving environment, rather than a single transitory game.

One approach to adding persistence to the NPSNET architecture is to associate an 'oldest client' with each multi-cast address. This is usually a process started when the world is first created and is responsible for storing the state of its particular area. As avatars move within range of a particular area, its current state can be requested from the relevant 'oldest client'.

### 2.4 Integrity

It is critical that all players can trust the integrity of the environment in which they are participating. Typical examples of integrity failures might be:

- players able to modify client code to provide them with a gaming advantage, e.g. turning off collision detection for their avatar, allowing them to move through walls,
- events and object behaviour depending on the quality of clients' network connections, e.g. objects disappearing or ceasing movement when a client's network connection fails,
- players able to modify locally stored state information to provide them with a gaming advantage, e.g. altering the effectiveness of weapons in their inventory.

Delivering consistency means ensuring all players perceive the same environment. Delivering integrity means ensuring that the environment behaves as players would expect. Without integrity the sense of immersion needed for a compelling game cannot be created.

This represents one of the key drawbacks to the NPSNET system (see section 2.2). With no centralised world management, responsibility for modelling behaviour is distributed to clients. This puts the integrity at risk, as clients are open to modification by the players. By contrast, the use of a central server helps deliver integrity, but at the risk of insufficient scalability.

## 2.5 Balance

Balance in a game is one of the essential factors to its success. A well-balanced game will always provide a challenge to the player, continually opening up new problems and new experiences, matching these to the current skill of the player. For example, in a single player game, the puzzles and hostile creatures within the game are gradually increased in toughness as the game progresses. Hence, as the player's skill level and the equipment available to their game character increases, the problems faced increase proportionally.

By contrast a multi-player game is far more complex to balance. For example:

- a new player may encounter a seasoned veteran within the first few minutes of play,
- the designer has little control over the state of the world encountered by any of the players after the first one has arrived,
- with a currency within the game a miniature economy may develop, and the presence of 'wealthy' players will impact on the behaviour of others,
- a particular weapon may prove too powerful, making the owner too significant an influence in the world.

To achieve balance in a persistent on-line environment, it may be necessary to introduce objects with new behaviour, or modify an existing object's behaviour, or introduce new rules governing the interaction between players; and since the environment is persistent, these changes must be made without closing the system down and re-compiling all the code.

## 2.6 Extensibility

If players are not to become bored with a virtual gaming world it is important to ensure that it can be extended. This means not only extending the volume of the world that can be explored, but also the range of objects within it and the behaviour of existing objects. One key feature of some MUDs is that players are rewarded for reaching a certain level of experience by being allowed to design new areas of the game. This not only acts as a spur to playing, but also

ensures the environment is regularly refreshed and redeveloped.

Having successfully built a community of users, extensibility offers a means to retain them.

## 3. DEE - distributed entertainment environment

DEE is an on-line gaming architecture being developed at BT Laboratories, designed to meet the requirements outlined in section 2. As such it draws on ideas from academic and military virtual environments, from MUDs, from Internet technologies and from existing games.

Its target network delivery platform is a BT product known as Wireplay.

### 3.1 Wireplay

In trying to deliver a shared gaming environment the performance of the network is critical, and the most important factor in the performance is latency. This is defined as the time period it takes information to propagate between nodes on the network. Without consistently low latency it is impossible to deliver highly interactive multi-player games.

To meet this particular network requirement BT developed Wireplay. This is designed to work with PCs using standard 28.8 kbit/s modems, and allows multi-player games to be played over guaranteed low-latency links (120 ms being the current client-to-client latency).

While Wireplay was initially targeted at supporting standard multi-player games, it is ideally suited for supporting virtual world gaming. This is because its architecture offers a centralised location suitable for placing servers capable of delivering the consistency, persistence and integrity discussed in section 2.

### 3.2 DEE — architecture overview

DEE has two distinguishing ideas. The first is concerned with the distribution of modelling behaviour within the VE, and is termed entity representation. The second is concerned with achieving scalability, and is termed dynamic management.

#### Entity representation

Anything that can be placed within the virtual environment is termed an entity. For example, a player's avatar is an entity, as is any equipment or scenery the player encounters.

While any entity represents a single concept, its behaviour and state can be grouped into three categories:

- visual — entities must be viewable, and have an interface with which players can interact (e.g. doors have handles allowing them to be opened),
- conceptual — entities incorporate game concepts, that relate to how they function within the designer specified rules (e.g. guns can be fired, which generate bullets; switches start machinery; medicine heals people, etc),
- dynamic — entities can move within the environment and collide with other entities.

DEE suggests that each of these categories is represented by a separate object, termed a representative, and that each entity is formed from the grouping of three of these representatives. For example, a sliding door in the environment might be represented by the triplet <DoorConcept, SlidingDoorDynamic, AnimatedVisual>. The DoorConcept representative would understand that doors can be opened by avatars, if they have the correct key. The SlidingDoorDynamic would be capable of calculating the movement as the door opened or closed, together with any associated collisions. The AnimatedVisual would be capable of rendering the correct geometry, and ensuring that as the door moved it was correctly animated.

In terms of placement within a network architecture, the various representatives would be arranged as shown in Fig 7.

Clients communicate dynamic change requests (e.g. move avatar, wave sword) to the dynamics manager (DM), and higher level game-related requests (i.e. open door, pull switch) to the rule manager (RM). The DM validates changes to the environment, updates the environment state and broadcasts the changes to clients. The RM responds to

requests by changing the game state and possibly placing requests with the DM (e.g. add velocity  $x$  to entity  $y$ ).

A typical messaging sequence for opening a door, might be:

- client requests action 'open' on its visual representation of a door — this generates a message from the visual representative to the conceptual representative (route 1 on Fig 8),
- the conceptual representation of the door at the RM decides if the player can perform the action (e.g. Is the player strong enough? Is the door unlocked? If it is locked, does the player have the key? etc),
- if the player can open the door, the door's conceptual representative will store the new state, and instruct the dynamic representative to swing the door open, thus generating a message from the conceptual representative to the dynamic representative (route 2 on Fig 8) — a typical message might contain just the request to open, or might also specify angle and speed criteria; the conceptual representative may also provide direct feedback to the client (e.g. 'You unlocked the door with the gold key'),
- the dynamic representative of the door, instantiated at the DM, will model the door movement, and broadcast changes to the client, thus generating a series of messages from the dynamic representative to the visual representatives (route 3 on Fig 8) — a typical set of messages might be 'start rotating at velocity  $x$ ' followed by 'stop rotation at angle  $y$ ',
- the visual representation of the door at each client will render its new appearance as it rotates into the open position (note that the visual representation should be capable of prediction and roll-back, so as to allow separation of the client rendering loop from the dynamic updates).

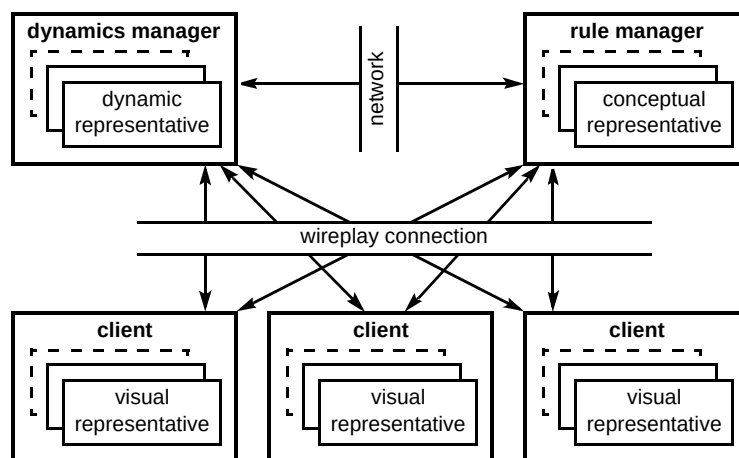


Fig 7 DEE architecture overview.

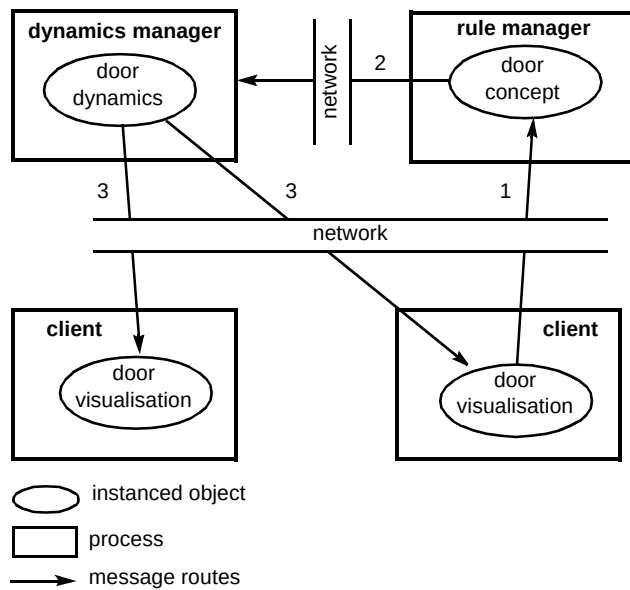


Fig 8 Message flow for opening a door.

- The centralisation of the games rules, in the conceptual representatives, guarantees integrity. The players have no access to the conceptual state, and hence cannot perform local modifications.
- The dynamic/conceptual separation aids extensibility. Developing conceptual representatives is a game-oriented task as the complexity related to visual/movement management has been removed. Hence, the game designer can focus on the rules and interaction of game concepts, rather than ensuring objects collide and render properly.
- The provision of persistence is possible, as all persistent state is held centrally. Standard persistent object technology can therefore be used to store the state of all dynamic/conceptual representative state, and hence the state of the VE [7].

### Dynamic management

While the concept of entity representation meets many of the requirements specified in section 2, it does not address the problem of scalability. Indeed, with a highly centralised, server-oriented architecture, it has potentially a serious scalability problem. DEE addresses this with a dynamic management system that provides efficient scaling of both the client network load and the server processing load.

It is assumed that the network and processing load associated with the conceptual representatives does not have a scalability problem. This is because players generate a large amount of network traffic related to the dynamics (moving, throwing objects, etc), but comparatively little in terms of game level actions (pulling switches, opening doors). This focuses the problem of scalability on to the dynamic manager.

DEE utilises an architecture similar to NPSNET, whereby the spatial environment is considered to be broken into predefined areas, termed zones. However, unlike NPSNET, responsibility for modelling state within these zones is not held by clients, but by processes termed zone managers. Each zone manager instances the dynamic representatives that fall within its zone, and is responsible for providing them with processing time to perform state updates. As dynamic representatives move they can be passed between zone managers, hence moving processing load and network traffic with the dynamic representative.

Each client determines the zones that it wishes to display locally and connects to the relevant zone manager. Each zone manager maintains a list of all connected clients, and as dynamic representatives update state (e.g. move,

### Advantages of entity representation

There are numerous advantages to dividing the modelling of the VE in the way suggested.

- The dynamic/conceptual separation allows the use of different technical implementations for the management of the VE. To facilitate **balance** the conceptual representatives should be implemented in a programmer-friendly, dynamically linkable, object-oriented language like Java [5]. This will allow the rules governing the environment to be easily modified. However, using Java for the dynamic representatives would be far too slow, as dynamic modelling is traditionally computationally expensive. Hence, a compiled language such as C++ is more suitable [6].
- The centralisation of the dynamic management guarantees consistency. Effectively, the dynamic representatives form a definitive description of the VE dynamic state, with the visual representatives shadowing this state as closely as possible. Requests to alter the dynamic state must be agreed by the dynamic representative, and since the dynamic state is updated centrally, unreliable clients will not impact on the overall consistency.
- The abstraction of the dynamics from the visual representation facilitates development of the client graphical capabilities. Not only is the client able to concentrate processing power on the visual effects, the separation simplifies redesigning visual representatives to make use of new graphics cards or APIs.

change velocity, etc), they inform all clients connected to their zone manager. In this way clients can constrain the network traffic to the areas of the environment in which they have an interest, while the dynamic representatives automatically ensure the relevant clients are kept informed of their state.

With potentially thousands of zones within any one VE it is not feasible to create a zone manager for every zone, as this would create too high a process management overhead. Instead when the state of a zone becomes dynamically deterministic (i.e. completely predictable over time), its state is written back into a persistent object store and the relevant zone manager closed. If new activity occurs within the zone (e.g. an avatar arrives, somebody throws an object into it, etc) a new zone manager is started and the previous zone state fetched. This will result in clusters of active zones (i.e. those with assigned zone managers) around the avatars in the environment, while most zones are dormant and held in the persistent store. Figure 9 shows an overall schematic for the system.

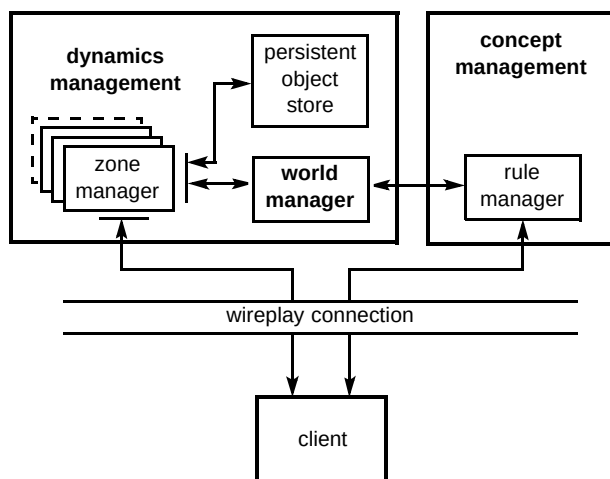


Fig 9 DEE process architecture.

Figure 9 has introduced a new process — the world manager. Its responsibility is to route messages between the rule manager and the zone managers (i.e. from conceptual representatives to dynamic representatives), the creation of zone managers, and load management. Creation of a zone manager occurs when either a request is placed from an existing zone (e.g. dynamic representative crossing zone boundaries) or when a conceptual representative tries to message to a dynamic representative in a dormant zone. In this situation the world manager creates a new zone manager process, and allocates the relevant key for it to extract the zone's current state from the persistent store. Coarse load management can be achieved by ensuring that the creation of the zone manager occurs on the least-loaded processor available.

It should be noted that the persistent object store shown in Fig 9 should be designed for highly optimised, rapid storage and retrieval of dynamic state only. Conceptual state resides in the rule manager, which should run for the life of the environment, with modifications made for balance and extensibility performed 'on the fly'.

If it were necessary to support closure and re-starting of the rule manager, and hence storage of conceptual representative state, a separate persistent object store should be provided. Since this is not a standard part of normal game operation (unlike zone state storage), speed is not an issue, and the implementation can be based on issues of simplicity and standardisation.

#### 4. Future work

The DEE architecture described here appears to offer a number of important ideas relevant to both VEMUDs and more general, shared virtual environments. It addresses all the major requirements outlined in section 2, while avoiding strict enforcement of a particular implementation technology.

An initial prototype is currently being developed at BT Laboratories, and work is under way on developing the test metrics and test harness that will allow an assessment of its potential. If the architecture proves its worth in testing, it is expected that game developers will be able to utilise it in delivering complex and scalable graphical environments over Wireplay.

#### Acknowledgements

The authors would like to gratefully acknowledge the help and support of Graham Walker, Tim Regan and Phil Sheppard.

#### References

- 1 Bartle R: 'Interactive multi-user computer games', <ftp://ftp.parc.xerox.com/pub/MOO/papers/mudreport.txt> (December 1990).
- 2 Lea R, Honda Y, Matsuda K, Rekimoto J: 'Technical issues in the design of a scalable shared virtual world', Sony Computer Science Lab, Tokyo, Japan (1990).
- 3 Morningstarr C and Farmer F: 'The lessons of Lucasfilm's Habitat', in 'Cyberspace: First Steps', MIT Press, Cambridge, Mass (1990).
- 4 Macedonia M R, Zyda M J, Michael J P, David R, Barham P T and Zeswitz S: 'NPSNET — a network software architecture for large scale virtual environments', *Presence*, 3, No 4 (Fall 1994).



- 5 Gosling J and McGilton H: 'The Java language environment. A White Paper', Sun Microsystems Computer Company, 2550 Garcia Avenue, Mountain View CA (October 1995).
  - 6 Stroustrup B: 'The C++ Programming Language', Addison-Wesley Publishing Company (June 1994).
  - 7 Saljoughy A: 'Object Persistence and Java', JavaWorld on-line, <http://www.javaworld.com/javaworld/jw-05-1997/jw-05-persistence.html> (May 1997).
- 



Mike Hinds received a BSc in computer science and an MSc in computer graphics from the University of East Anglia before becoming a research associate at the university where he worked on functional languages and parallel graph reduction. He joined BT in 1991 working in the Data Visualisation group implementing prototype visualisations of the vast amounts of network data the company collects. The visualisation work has been carried on into the shared information environments project where he is currently researching large multi-user persistent environments.



Simon Powers graduated with an MEng in Electronic Systems from York University, UK in 1994. He joined the research department of BT Laboratories to work on virtual environments and human/computer interfaces.

Recent work has concentrated on shared environments, in particular looking at suitable application architectures for the entertainment/computer gaming industry.



Jason Morphet completed his BSc in Software Engineering at Suffolk College (associate college of the UEA) following an HND in the same subject. He is currently a registered PhD student at the University of Nottingham. He joined BT as a summer student in June 1995, staying on to become a Research Fellow and subsequently a full-time employee. His interests include both the network and human interface issues of distributed virtual environments, and he was closely involved in the 'Portal' development.